Unit 1: Number Systems and Data Representation

Digital systems operate on binary data. This unit covers the mathematical foundations required to represent and manipulate data within a computer architecture.

1.1 Number Base Conversions

Number systems are defined by their base or radix (r). The most common systems in digital electronics are Decimal (r=10), Binary (r=2), Octal (r=8), and Hexadecimal (r=16).

- **Decimal to Binary/Octal/Hex (Integer Part):** Use the repeated division method. Divide the decimal number by the target base and record the remainders. Read remainders from bottom (MSB) to top (LSB).
- **Decimal to Binary/Octal/Hex (Fractional Part):** Use the repeated multiplication method. Multiply the fractional part by the target base, record the integer overflow, and repeat with the new fraction. Read from top to bottom.
- **Binary/Octal/Hex to Decimal:** Multiply each digit by its positional weight (base raised to the power of its position) and sum the results.
- **Octal and Hexadecimal Shortcuts:** Since $8 = 2^3$ and $16 = 2^4$, each octal digit corresponds exactly to a 3-bit binary group, and each hex digit corresponds to a 4-bit binary group.

1.2 Binary Arithmetic and Complements

Direct subtraction requires complex circuitry. Digital systems utilize complements to perform subtraction via addition.

- **1's Complement:** Invert all bits (0 becomes 1, 1 becomes 0). Subtraction involves adding the 1's complement of the subtrahend. If a carry is generated out of the MSB, it must be added back to the LSB (End-Around Carry).
- **2's Complement:** The 1's complement plus 1. This is the standard for signed arithmetic. Subtraction involves adding the 2's complement of the subtrahend. Any carry generated out of the MSB is simply discarded.
- **Sign Extension:** To expand a signed binary number to more bits (e.g., 4-bit to 8-bit), replicate the Sign Bit (MSB) into the newly added higher-order positions.

1.3 Digital Codes and Error Correction

Data must be encoded for transmission, computation, and display.

- **Weighted Codes:** BCD (8421), 2421, and 84-2-1. Used when positional weights map directly to values.
- **Non-Weighted Codes:** Excess-3 (add 0011 to BCD) and Gray Code. Gray code has the property that only one bit changes between successive numbers, minimizing transition errors.
- **Alphanumeric Codes:** ASCII (7-bit) and EBCDIC (8-bit) represent characters, numbers, and control signals.
- **Error Detection and Correction:**
    - *Parity Bit:* Detects single-bit errors.
    - *Hamming Code:* Uses multiple parity bits interspersed with data bits to both detect and locate single-bit errors, allowing for automatic correction.

Unit 2: Boolean Algebra and Logic Minimization

Minimizing logic expressions reduces the hardware complexity, latency, and power consumption of a digital circuit.

2.1 Universal Gates Realization

NAND and NOR are universal because they can emulate any other gate.

- **NOT from NAND:** Tie both inputs of a 2-input NAND gate together.
- **AND from NAND:** Use one NAND gate to generate (AB)', then pass it through a second NAND gate configured as a NOT.
- **OR from NAND:** Invert inputs A and B individually using NAND-NOTs, then pass A' and B' into a third NAND gate. By De Morgan's: $(A' \cdot B')' = A + B$.

2.2 Standard Forms and K-Maps

- **Canonical SOP and POS:** An expression is in canonical form if every term contains all the variables of the function. SOP uses Minterms (m), POS uses Maxterms (M).
- **Karnaugh Maps (Up to 5 Variables):** A visual matrix representing a truth table. A 4-variable map has 16 cells. A 5-variable map uses two 16-cell maps layered virtually on top of each other. Groupings (pairs, quads, octets) must be powers of 2.
- **Don't Care Conditions (X):** Outputs for certain input combinations that will never naturally occur. They are strategically grouped with 1s (for SOP) to form larger, simpler loops, or ignored if unhelpful.
- **Quine-McCluskey (Tabular) Method:** An algorithmic approach to minimization that is better suited for functions with more than 4 variables or for computer automation. It involves finding Prime Implicants and then selecting Essential Prime Implicants using a coverage table.

Unit 3: Combinational Logic Design

Combinational logic consists of interconnected logic gates where the output is determined strictly by the present inputs.

3.1 Advanced Arithmetic Circuits

- **Ripple Carry Adder:** Cascades N Full Adders to add N-bit numbers. The carry out of one stage is the carry in of the next. It is slow due to carry propagation delay.
- **Carry Look-Ahead Adder (CLA):** Reduces computation time by pre-calculating carry signals using Generate ($G = A \cdot B$) and Propagate ($P = A \oplus B$) functions, rather than waiting for ripples.
- **BCD Adder:** Adds two BCD digits. If the binary sum exceeds 9 (1001) or a carry is generated, it corrects the result by adding 6 (0110) to skip the invalid BCD states.

3.2 Comparators and Data Routers

- **Magnitude Comparator:** Compares two binary numbers A and B and outputs three signals: $A > B$, $A < B$, and $A = B$. It utilizes XNOR gates heavily for equality checking.

- **Multiplexers (MUX) as Logic Implementers:** A 2^n-to-1 MUX can implement any Boolean function of 'n+1' variables. The first 'n' variables are connected to the selection lines, and the remaining variable (or constants 0, 1) is wired to the data inputs based on the truth table.
- **Decoders in Logic Design:** An n-to-2^n decoder generates all possible minterms of 'n' variables. By connecting the required minterm outputs to an OR gate, any combinational circuit can be realized without minimizing equations.

Unit 4: Sequential Logic Circuits

Sequential circuits incorporate memory, enabling systems to retain state over time. These concepts form the architectural basis for complex, timing-dependent projects like a digital password lock system, which must "remember" the sequence of inputted bits to grant access.

4.1 Flip-Flop Fundamentals and Tables

To design sequential circuits, you must master the transition behaviors of flip-flops.

- **Characteristic Table:** Defines the next state Q(t+1) based on the present inputs and the present state Q(t).
- **Excitation Table:** Crucial for design. It works backward: given the present state Q(t) and desired next state Q(t+1), what inputs must be applied to the flip-flop to make that transition happen?

Transition Q(t) → Q(t+1)	SR Flip-Flop (S, R)	JK Flip-Flop (J, K)	D Flip-Flop (D)	T Flip-Flop (T)
0 → 0	0, X	0, X	0	0
0 → 1	1, 0	1, X	1	1
1 → 0	0, 1	X, 1	0	1
1 → 1	X, 0	X, 0	1	0

4.2 Race Around Condition & Master-Slave

- **The Problem:** In a standard JK flip-flop, if J=1, K=1, and the clock pulse width is longer than the propagation delay of the flip-flop, the output will toggle unpredictably multiple times during a single clock pulse. This is the race-around condition.
- **The Solution: Master-Slave JK.** Consists of two cascaded flip-flops. The Master triggers on the positive clock edge, while the Slave (due to an inverted clock) triggers on the negative edge. This isolates the output from the inputs, ensuring the state only changes once per full clock cycle.

4.3 Flip-Flop Conversion

Any flip-flop can be converted into another by adding external combinational logic to its inputs.

- **Method:** Write the characteristic table of the *target* flip-flop. Use the excitation table of the *available* flip-flop to determine the required inputs. Map these inputs into a K-map relative to the target's inputs and present state Q(t) to derive the combinational logic circuit.

Unit 5: Registers, Counters, and Memory Devices

These complex sequential circuits scale up individual flip-flops into subsystems capable of holding bytes/words of data and executing state-machine behavior.

5.1 Shift Register Applications

- **Data Delay:** A SISO shift register delays a digital signal by 'n' clock cycles.
- **Multiplication/Division:** Shifting binary data to the left by one bit multiplies it by 2. Shifting to the right divides it by 2.
- **Universal Shift Register:** A versatile bidirectional register equipped with a mode control input that determines whether it operates as PIPO, SISO, left-shift, right-shift, or hold

state.

5.2 Counter Design Protocols

- **Modulo-N Asynchronous Counter:** A counter that resets before reaching its natural limit (e.g., Mod-10 / Decade counter). If counting from 0 to 9, the counter forces a reset when it reaches 10 (binary 1010). A NAND gate detects the 1s in the 1010 state and drives the asynchronous CLR (clear) inputs of the flip-flops.
- **Synchronous Counter Design Steps:**
    1. Determine the number of flip-flops required ($2^n \geq$ number of states).
    2. Draw the State Transition Diagram mapping the sequence.
    3. Construct a State Table listing Present State and Next State.
    4. Expand to an Excitation Table based on the chosen flip-flop type (usually JK or T).
    5. Use K-maps to minimize the logic equations for each flip-flop's inputs.
    6. Draw the final logic diagram with a common clock line.

5.3 Programmable Logic Devices (PLD)

PLDs provide a flexible method for implementing complex combinational circuits without wiring discrete chips.

- **PROM (Programmable ROM):** Fixed AND array (decoder) and a Programmable OR array. Ideal for implementing standard Sum of Products where all minterms are generated.
- **PAL (Programmable Array Logic):** Programmable AND array and a Fixed OR array. More efficient and faster than PROM, used for standard Boolean minimization logic.
- **PLA (Programmable Logic Array):** Both the AND array and the OR array are programmable, making it the most flexible, but also the slowest and most complex to manufacture and program.